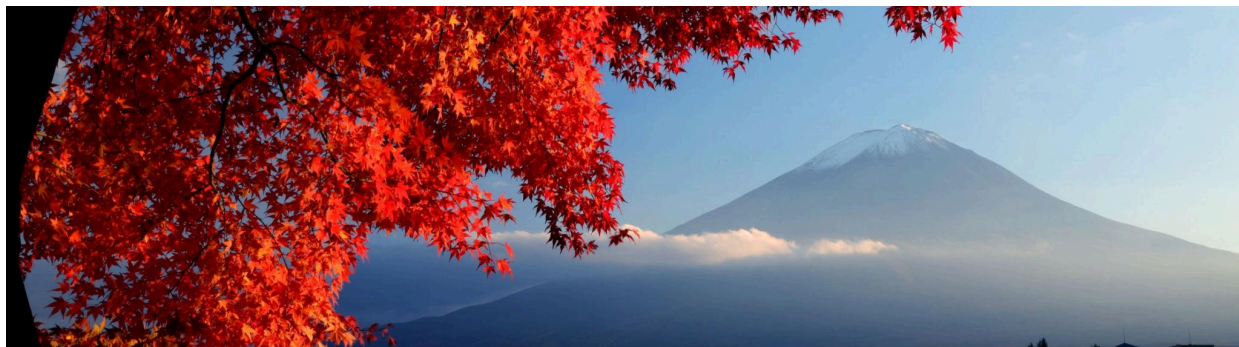




Train Your Own Small Learning Model

Research it, generate the data, fine-tune it, prove it works. One week. You'll take a small open base model and instill a specific learning or teaching behavior of your choosing — end to end. The point isn't to out-smart a frontier model. It's to prove you can make a model *reliably do one thing* by controlling its training data.

The one idea to internalize first

The dataset is the deliverable, not the model. In a one-week build, ~80% of outcome quality is the data you generate. Training is a downstream button-press. The model is just your data made runnable.

So "train" here does **not** mean pretraining from scratch... A week of that produces noise and teaches you nothing but CUDA pain. It means **supervised fine-tuning (QLoRA) on a small open base model**, using data you generate and filter.

What you build is open — with one hard test

You choose what your model does. But the behavior you pick has to pass one litmus test:

A well-prompted base model can't already do it reliably.

That test is the whole game. If a good prompt already nails your target, fine-tuning is pointless — just prompt the base model. Fine-tuning earns its place only when **reliability** is the hard part: the model must do the thing *every time*, in-character, without drifting. That's what a dataset buys you and a prompt can't guarantee.

So the free variable is the behavior; the fixed constraint is that it's a behavior worth *training* rather than prompting.

The gate: write a Behavior Spec before anything else

"A model that does X" is too vague to train toward or grade. Your first deliverable is a **falsifiable behavioral spec** — one or two sentences a stranger could use to mark any model output pass/fail.

Example (tutor):

The model never states the final answer. Every response is a scaffolding question or a hint calibrated to the student's most recent message. It only confirms an answer once the student produces it themselves.

Example (structured-output):

The model always returns a single valid JSON object matching the given schema, with no prose before or after, even when the input is incomplete or adversarial.

That one spec is simultaneously your **data-generation rubric**, your **eval criterion**, and your **brainlift spiky POV**. Everything downstream serves it.

Where the real work is

Two things carry this project. Neither is the training loop.

1. Data generation. Distill from a frontier "teacher" model: generate hundreds to a few thousand examples that embody your spec, then filter hard for quality. AI costs are covered.. The craft is in the generation prompt and the quality gate, not raw volume.

2. Evaluation — built before you train. This is the make-or-break, and the piece everyone skips. Without an eval, "we fine-tuned a model" is unfalsifiable. Minimum bar:

- An LLM-as-judge scoring outputs against your Behavior Spec
- A behavioral check for the specific failure your spec forbids
- A **base-vs-tuned comparison** so your fine-tune's effect is visible in numbers

If you can't measure that your tuned model beats the base at your target behavior, you haven't finished.

A note on "why does this even exist"

Your 1B model will **not** beat a frontier model on raw capability. If you benchmark it that way, you'll feel like you failed. You measured the wrong thing.

The defensible win is *reliable, constrained behavior in a tiny, cheap, local model* — one narrow thing it does consistently that's genuinely hard to get reliably from prompting. Frame your thesis as "behavior from data," not "smarter than GPT." (Fine-tuning small open models into reliable specialists that rival prompted frontier models is a proven method right now, not a moonshot.)

One-week arc

Day	Focus	Actions	Checkpoint
1	Setup, research, & Brainlift	Get the environment working to run inference. Research behavior. Complete your Brainlift.	The base model runs and responds; target behavior is known. Spiky POVs match target behavior.
2	Spec, eval, & smoke test	Write Behavior Spec. Build eval harness and data-gen pipeline. Run 50 junk examples.	Full loop (generate → train → eval) runs end to end.
3	v1 dataset & real numbers	Generate and filter real data. First real training run. First base-vs-tuned eval.	Midweek gate: base-vs-tuned numbers are on the board.
4	v2 dataset (iteration)	Diagnose failure modes. Fix in data, not hyperparameters. Retrain and report improvements.	One specific failure mode resolved via data iteration.
5	Ship & defend	Final eval + error analysis. Ship inference demo. Write brainlift and record demo.	Final submission package ready.

Stretch ladder

Finishing the core arc early means going *deeper*, not idling. Climb this in roughly this order — each rung is a real, gradeable result on its own. DPO and the adversarial eval are the natural first two.

1. **DPO / preference tuning.** Build preference pairs (on-spec vs. off-spec responses) and run DPO on top of your SFT model. Measure whether it sharpens spec adherence beyond SFT alone. (*Deepens the training technique.*)
2. **Adversarial / robustness eval.** Build a hard eval set designed specifically to *break* your behavior — jailbreak the tutor into giving answers, feed malformed input to the schema model. Report robustness under attack, not just clean inputs. (*Deepens evaluation.*)
3. **Composed behavior.** Instill a second, potentially competing constraint and show the model holds both (e.g. never gives answers *and* stays encouraging). Tests whether data can encode multiple behaviors at once. (*Hardest — competing constraints.*)

Final submission package

1. **The dataset**, published (this is your real artifact)
2. **The model** on Hugging Face Hub + a running inference demo
3. **Eval harness + results table** — base vs tuned, with your behavior metric
4. **Brainlift** — your behavior thesis and whether data→behavior held, with evidence
5. **3–5 min demo video** showing the model doing the thing the base model *fails* to do reliably

Stack Suggestions

- **Base model**: a small **Qwen3** (0.6B / 1.7B / 4B) is the current default. Alternates: Llama 3.2 1B/3B, Gemma 3 small, SmolLM3. Start from the Instruct variant for fast SFT.
- **Framework**: **Unsloth** for QLoRA — ~2× faster, ~70% less VRAM, clean notebooks, single GPU. (TRL/PEFT or Axolotl for more control.)
- **Compute**: one A100/H100 via Modal / RunPod / Colab. Models ≤1.7B fit a 24GB consumer card.
- **Teacher (for distillation)**: any frontier model — costs covered.

Rules / traps to avoid

- **Pick a behavior that fails the prompt test**. If a good prompt on the base model already does it reliably, you've picked a bad target.
- **No training before the eval exists**. Left alone you'll fine-tune on day 1, get something plausible-sounding, and have no way to know if it beats the base.
- **No broad domains**. One target, one context. Diffuse data makes a mushy model.
- **Don't tune hyperparameters to fix a data problem**. Data is the lever. Nine times out of ten a disappointing model is a data problem.
- **Don't chase capability benchmarks**. Measure your target behavior, not trivia accuracy.

Appendix A — Eval rubric example (fork this)

Score each model output (base and tuned) with an LLM-as-judge. Report the delta.

Dimension	0	1	2
Spec adherence	Violates the target behavior	Partially follows	Fully embodies the spec
Robustness	Breaks on messy/adversarial input	Wobbles	Holds the behavior under pressure
Task quality	Output is wrong or useless	Acceptable	Genuinely good at the job

Dimension	0	1	2
Consistency	Behaves differently across similar inputs	Mostly stable	Reliable every time

Required outputs:

- Mean score per dimension, **base vs. tuned**, on the same held-out scenarios
- A short error-analysis paragraph: where does the tuned model still fail, and is it a data problem?

A tuned model that beats the base on *Spec adherence* and *Robustness* is a win.